

Image Generation Using DCGAN

Project submitted to the
SRM University – AP, Andhra Pradesh
for the partial fulfilment of the requirements to award the degree of

Bachelor of Technology
In
Computer Science and Engineering
School of Engineering and Sciences

Submitted by
M.Lalith Reddy (AP22110011442)
D.Sai Bhargav (AP22110011465)
T.Harshit(AP22110011476)
Y.Pavan(AP22110011474)



Under the Guidance of
Abhijit Dasgupta
SRM University–AP
Neerukonda, Mangalagiri, Guntur
Andhra Pradesh – 522240

CERTIFICATE

Date: 28-Nov-24

This is to certify that the work presented in this project, entitled Image generation using DCGAN has been carried out by Lalith, Pavan, Harshit and Bhargav under my/our supervision. The work is genuine, original, and suitable for submission to SRM University–AP for the award of Bachelor of Technology in the School of Engineering and Sciences.

Supervisor

(Signature)

Prof. Abhijit Dasgupta

Designation, Affiliation.

ACKNOWLEDGEMENTS

We extend our sincere gratitude to the organizations and institutions that provided the datasets and resources essential for this project. Special thanks to our mentors, Abhijit Dasgupta, and our advisors for their invaluable guidance and constructive feedback throughout the development process.

We acknowledge the contributions of researchers and developers whose pioneering work in generative adversarial networks and deep learning laid the foundation for advancements in image generation.

Additionally, we are grateful for the availability of open-source tools and libraries, which made model implementation and experimentation feasible.

The collaboration and shared expertise within our team were integral to this project's success. Finally, we recognize the role of evolving deep learning frameworks, which enabled the development of robust and efficient DCGAN models across different datasets.

TABLE OF CONTENTS

CONTENT	PAGE NUMBER
Abstract	5
Problem statement	6
Introduction	7
Scope of study and limitations	8
Data set description	9
Methodology	12
Results and analysis	17
Conclusion	20

ABSTRACT

This project explores the application of Deep Convolutional Generative Adversarial Networks (DCGANs) for generating realistic images from random noise. DCGANs, a class of generative models, leverage convolutional neural networks (CNNs) to create high-quality synthetic images. The project focuses on three distinct datasets: MNIST (handwritten digits), SVHN (street view house numbers), and CelebA (celebrity faces), aiming to investigate the ability of DCGANs to model complex data distributions and generate visually plausible images.

The methodology involves training separate DCGAN models for each dataset, where the generator and discriminator networks are iteratively optimized through adversarial training. The project evaluates the effectiveness of the models based on the visual quality of the generated images, as well as the impact of dataset complexity on model performance. Results demonstrate that DCGANs can generate highly realistic images for simpler datasets like MNIST, while generating more detailed and complex images for datasets like CelebA requires deeper networks and longer training periods. This project provides insights into the challenges and capabilities of DCGANs in the domain of image synthesis.

Problem Statement: Image Generation Using DCGAN

The goal of this project is to implement and train Deep Convolutional Generative Adversarial Networks (DCGANs) to generate realistic images from random noise vectors. DCGANs, a variant of the traditional Generative Adversarial Network (GAN), utilize convolutional layers to improve the quality and stability of generated images. The project focuses on training DCGAN models on three distinct image datasets — MNIST (handwritten digits), SVHN (street view house numbers), and CelebA (celebrity faces).

The primary objective is to evaluate the effectiveness of DCGANs in generating images that closely resemble real-world data. Specifically, the project aims to:

- Implement a robust DCGAN architecture using deep convolutional networks.
- Train the DCGAN models on different datasets with varying levels of complexity.
- Assess the quality of generated images through visual inspection and training metrics.
- Analyze how dataset complexity impacts model performance and training stability.

The problem to be solved is to understand how DCGANs perform on a range of image generation tasks and to determine the challenges and limitations associated with generating complex images, especially for larger and more varied datasets.

Introduction

Generative Adversarial Networks (GANs) are a class of deep learning models that have revolutionized the field of generative modeling, particularly in image generation. Introduced by Ian Goodfellow in 2014, GANs consist of two competing neural networks: a generator and a discriminator. The generator creates synthetic images from random noise, while the discriminator evaluates whether these images are real (from the training data) or fake (generated by the model). Through this adversarial training process, both networks improve, with the generator progressively learning to produce more realistic images and the discriminator becoming more skilled at distinguishing real from fake.

Deep Convolutional GANs (DCGANs) are a variant of GANs that leverage convolutional layers in both the generator and discriminator networks. This architecture improves the quality of the generated images by using convolutional operations that preserve spatial hierarchies in images, making it more effective for generating complex visual data. DCGANs have been widely used in various applications such as generating faces, artwork, and even photorealistic images.

This project focuses on the application of DCGANs to generate realistic images using three different datasets: MNIST (handwritten digits), SVHN (street view house numbers), and CelebA (celebrity faces). Each of these datasets presents unique challenges, from the simplicity of MNIST's digit images to the complexity of CelebA's detailed human faces. The project aims to explore the following:

- The effectiveness of DCGANs in generating images across datasets of varying complexity.
- The challenges in training GANs for different types of data, ranging from simple, well-structured images (MNIST) to more diverse and complicated datasets (SVHN and CelebA).

An in-depth evaluation of the visual quality of generated images through qualitative analysis and performance metrics.

Study and Limitations

Scope:

- Development of an end-to-end deep learning pipeline for image denoising using Convolutional Autoencoders.
- Application of the model on the MNIST dataset with artificially added Gaussian noise.
- Evaluation of model performance using both qualitative (image visualization) and quantitative (MSE) metrics.
- Exploration of training dynamics and loss behavior over epochs.

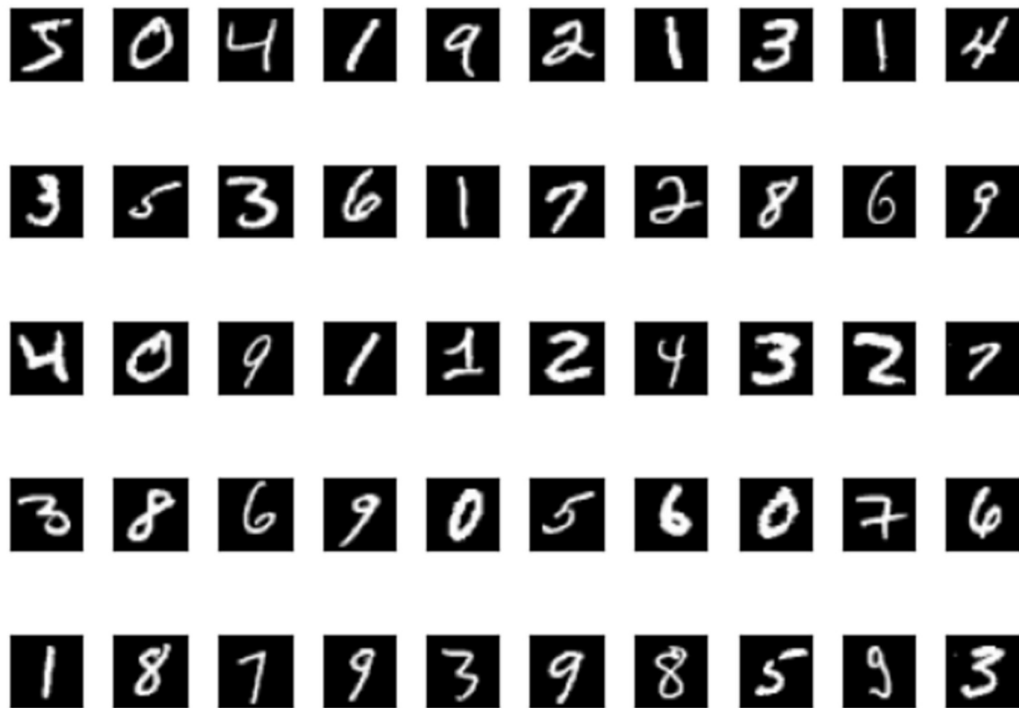
Limitations:

- The model is trained only on 28x28 grayscale images (MNIST); performance on high-resolution or colored images is not evaluated.
- Only Gaussian noise is considered; the model may require re-training or fine-tuning for other noise types such as Salt-and-Pepper or Speckle noise.
- The simplicity of the MNIST dataset may not reflect the challenges posed by real-world noisy image data.
- The model architecture is relatively shallow and could be improved with deeper or more complex networks like U-Net or ResNet-based encoders.

Dataset Description

3.1 MNIST Dataset:

The MNIST dataset consists of 70,000 grayscale images of handwritten digits (0-9), with 60,000 images in the training set and 10,000 in the test set. Each image is 28x28 pixels in size, and the dataset provides a simple task for evaluating the performance of GAN models, as the digits are clearly structured and well-separated.



3.2 SVHN Dataset:

The SVHN dataset contains over 600,000 labeled images of house numbers taken from Google Street View. Each image is 32x32 pixels and consists of digits (0-9) displayed in various real-world conditions. This dataset is more challenging due to varying backgrounds, lighting, and the presence of other elements in the images.



3.3 CelebA Dataset:

CelebA is a large dataset containing over 200,000 celebrity face images, each labeled with 40 attribute labels such as age, gender, and facial expressions. The images are 64x64 pixels and are cropped to focus on faces. This dataset is significantly more complex than MNIST and SVHN, requiring the generation of highly detailed and diverse facial features.



Methodology

4.1 Data Preparation

The first step in building the DCGAN models involved collecting and preprocessing the datasets. The MNIST, SVHN, and CelebA datasets were used to train different versions of the model:

MNIST Dataset: Consists of 28x28 grayscale handwritten digits. Images were normalized between -1 and 1 to match the tanh activation output.

SVHN Dataset: Contains 32x32 colored images of house numbers captured from street scenes. Similar normalization was applied.

CelebA Dataset: Contains celebrity face images resized to 64x64 pixels. Each image was also normalized between -1 and 1.

Preprocessing was crucial for ensuring consistent input size and pixel value ranges, allowing the GAN models to train stably.

4.2 Model Design

The Deep Convolutional Generative Adversarial Networks (DCGANs) used in this project consist of two core components:

A Generator network that tries to create realistic-looking images.

A Discriminator network that attempts to distinguish between real and generated (fake) images.

Separate architectures were designed for the MNIST, SVHN, and CelebA datasets to match the complexity of the images.

4.2.1 Generator Architecture

The Generator is responsible for transforming a random noise vector into a realistic image.

Each dataset has its own slightly customized Generator:

For MNIST Dataset:

```
def make_generator(input_size, leaky_alpha):
    # generates images in (28,28,1)
    return Sequential([
        # FC 1: 7,7,16
        Dense(784, input_shape=(input_size,)),
        Reshape(target_shape=(7, 7, 16)),
        BatchNormalization(),
        LeakyReLU(alpha=leaky_alpha),

        # Conv 1: 14,14,32
        Conv2DTranspose(32, kernel_size=5, strides=2, padding='same'),
        BatchNormalization(),
        LeakyReLU(alpha=leaky_alpha),

        # Conv 2: 28,28,1
        Conv2DTranspose(1, kernel_size=5, strides=2, padding='same'),
        Activation('tanh')
    ])
```

For SVHN Dataset:

```
def make_generator(input_size, leaky_alpha):
    # generates images in (32,32,3)
    return Sequential([
        Dense(4*4*512, input_shape=(input_size,)),
        Reshape(target_shape=(4, 4, 512)),
        BatchNormalization(),
        LeakyReLU(alpha=leaky_alpha),
        Conv2DTranspose(256, kernel_size=5, strides=2, padding='same'),
        BatchNormalization(),
        LeakyReLU(alpha=leaky_alpha),
        Conv2DTranspose(128, kernel_size=5, strides=2, padding='same'),
        BatchNormalization(),
        LeakyReLU(alpha=leaky_alpha),
        Conv2DTranspose(3, kernel_size=5, strides=2, padding='same'),
        Activation('tanh')
    ])
```

For CelebA Dataset:

```
def make_generator(input_size, leaky_alpha, init_stddev):
    # generates images in (32,32,3)
    return Sequential([
        Dense(4*4*512, input_shape=(input_size,),
            kernel_initializer=RandomNormal(stddev=init_stddev)),
        Reshape(target_shape=(4, 4, 512)),
        BatchNormalization(),
        LeakyReLU(alpha=leaky_alpha),
        Conv2DTranspose(256, kernel_size=5, strides=2, padding='same',
            kernel_initializer=RandomNormal(stddev=init_stddev)),
        BatchNormalization(),
        LeakyReLU(alpha=leaky_alpha),
        Conv2DTranspose(128, kernel_size=5, strides=2, padding='same',
            kernel_initializer=RandomNormal(stddev=init_stddev)),
        BatchNormalization(),
        LeakyReLU(alpha=leaky_alpha),
        Conv2DTranspose(3, kernel_size=5, strides=2, padding='same',
            kernel_initializer=RandomNormal(stddev=init_stddev)),
        Activation('tanh')
    ])
```

4.2.2 Discriminator Architecture:

The Discriminator acts as a binary classifier that predicts whether an input image is real (from the dataset) or fake (generated).

For MNIST Dataset

```
def make_discriminator(leaky_alpha):
    # classifies images in (28,28,1)
    return Sequential([
        # Conv 1: 14,14,32
        Conv2D(32, kernel_size=5, strides=2, padding='same', input_shape=(28,28,1)),
        LeakyReLU(alpha=leaky_alpha),

        # Conv 2: 7,7,16
        Conv2D(16, kernel_size=5, strides=2, padding='same'),
        BatchNormalization(),
        LeakyReLU(alpha=leaky_alpha),

        # FC 1
        Flatten(),
        Dense(784),
        BatchNormalization(),
        LeakyReLU(alpha=leaky_alpha),

        # Output
        Dense(1),
        Activation('sigmoid')
    ])
```

For SVHN Dataset:

```
def make_discriminator(leaky_alpha):
    # classifies images in (32,32,3)
    return Sequential([
        Conv2D(64, kernel_size=5, strides=2, padding='same',
            input_shape=(32,32,3)),
        LeakyReLU(alpha=leaky_alpha),
        Conv2D(128, kernel_size=5, strides=2, padding='same'),
        BatchNormalization(),
        LeakyReLU(alpha=leaky_alpha),
        Conv2D(256, kernel_size=5, strides=2, padding='same'),
        BatchNormalization(),
        LeakyReLU(alpha=leaky_alpha),
        Flatten(),
        Dense(1),
        Activation('sigmoid')
    ])
```

For CelebA Dataset:

```
def make_discriminator(leaky_alpha, init_stddev):
    # classifies images in (64,64,3)
    return Sequential([
        Conv2D(64, kernel_size=5, strides=2, padding='same',
              kernel_initializer=RandomNormal(stddev=init_stddev),
              input_shape=(32, 32, 3)),
        LeakyReLU(alpha=leaky_alpha),
        Conv2D(128, kernel_size=5, strides=2, padding='same',
              kernel_initializer=RandomNormal(stddev=init_stddev)),
        BatchNormalization(),
        LeakyReLU(alpha=leaky_alpha),
        Conv2D(256, kernel_size=5, strides=2, padding='same',
              kernel_initializer=RandomNormal(stddev=init_stddev)),
        BatchNormalization(),
        LeakyReLU(alpha=leaky_alpha),
        Flatten(),
        Dense(1, kernel_initializer=RandomNormal(stddev=init_stddev)),
        Activation('sigmoid')
    ])
```

4.3 Training Process

The DCGAN models were trained following an adversarial process:

Noise Generation: Random noise vectors were fed into the Generator to produce fake images.

Discriminator Training: The Discriminator was trained on batches containing real images (labeled as real) and fake images (labeled as fake).

Generator Training: The Generator was updated through the Discriminator's feedback, aiming to make generated images more realistic.

Loss Functions:

Binary cross-entropy loss was used for both Generator and Discriminator.

Optimizers: Adam optimizer with a learning rate of 0.0002 and beta1 of 0.5.

Training was conducted over several epochs, and generated images were saved at intervals to monitor the progress.

4.4 Evaluation Metrics

The performance of the DCGAN models was evaluated through the following methods:

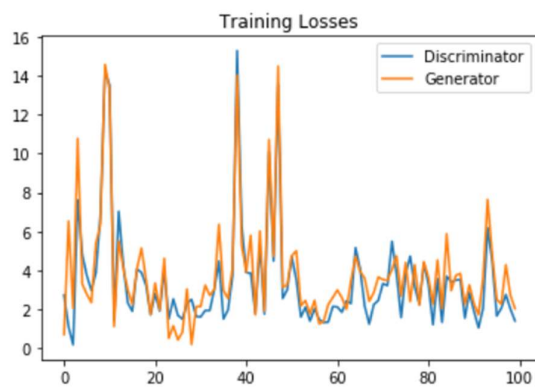
Visual Inspection:

Generated images were visually reviewed at different training stages to observe improvements in quality, sharpness, and realism.

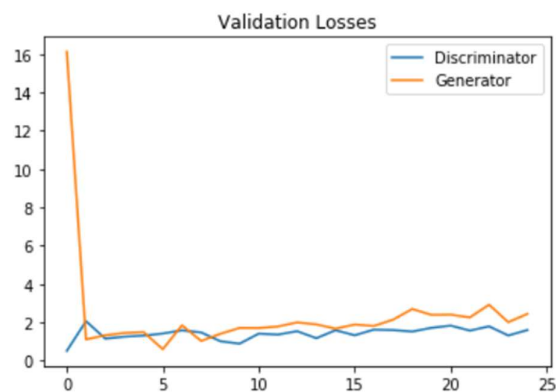
Training and Validation Loss Curves:

Both Generator and Discriminator loss values for training and validation sets were plotted across epochs. Monitoring these curves helped identify overfitting, underfitting, and training stability issues. A balanced decrease in both training and validation losses indicates good model convergence.

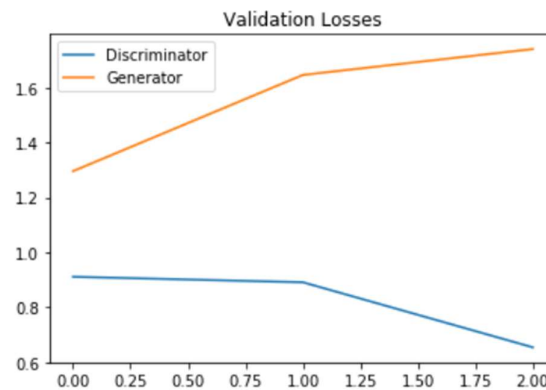
For MNIST:



For SVHM:



For CelebA:



Qualitative Analysis:

The generated samples were qualitatively assessed based on:

Visual clarity

Diversity across samples

Reduction of artifacts over epochs

This combined approach ensured that both the quantitative behavior (loss plots) and qualitative outputs (images) were evaluated comprehensively.

Results and Analysis

After training the DCGAN models on the MNIST, SVHN, and CelebA datasets, the Generator networks progressively improved in their ability to produce realistic images. The following results were observed for each dataset:

5.1 MNIST Results:

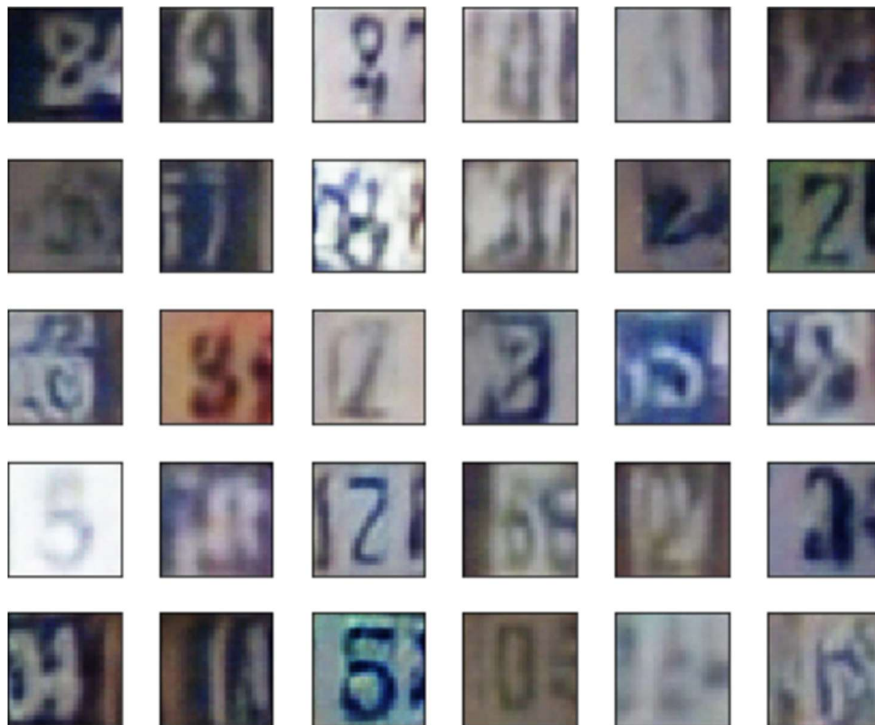
- The model was able to generate clear, sharp handwritten digits that closely resembled the original MNIST samples.
- During the initial epochs, the images were blurry and distorted, but as training progressed, the quality improved significantly.
- The Generator successfully captured the different styles of handwritten digits.



Output: Generated MNIST images after training.

5.2 SVHN Results:

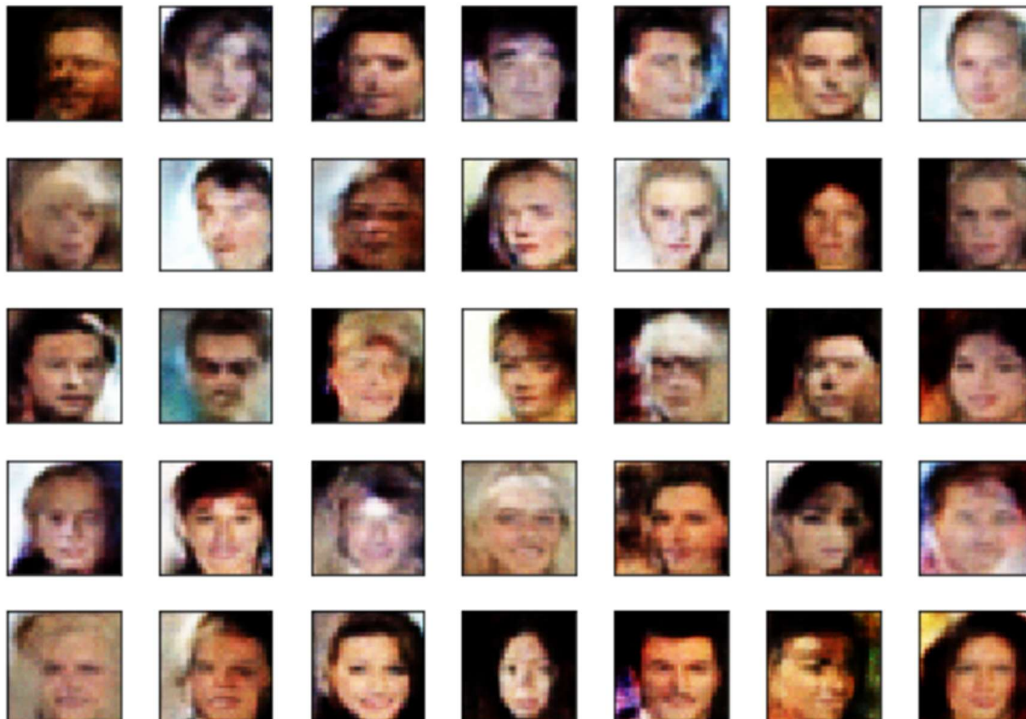
- SVHN was a more complex dataset due to colored and real-world images.
- Initially, the generated images contained noise and lacked proper structure.
- Over training epochs, the Generator produced colorful and well-structured images resembling real house numbers.
- Some artifacts persisted but reduced with longer training.



- **Output:** Generated SVHN images after training.

5.3 CelebA Results:

- CelebA presented the most challenging scenario due to higher image resolution and variability (different faces, poses, lighting conditions).
- Early epochs showed noisy and unrealistic outputs.
- After extensive training, the Generator learned to produce human-like faces with distinguishable features such as eyes, nose, and mouth.
- Some outputs were very sharp, while others had minor blurring or incomplete features.



- Output: Generated CelebA images after training.

Conclusion

In this project, Deep Convolutional Generative Adversarial Networks (DCGANs) were successfully implemented and trained on three different datasets:

MNIST, SVHN and CelebA .The models demonstrated the ability to generate realistic images across varying levels of complexity — from simple handwritten digits to colored house numbers and complex human faces.Training and validation loss curves provided insights into the stability and convergence of the networks. Visual inspections of the generated outputs highlighted the progressive improvement of image quality with training.Through this project, the effectiveness of GAN-based architectures in synthetic image generation was clearly observed.

Despite minor challenges like artifacts and blurring in more complex datasets, the results confirmed that DCGANs can be powerful tools for unsupervised learning tasks.This work lays the foundation for future improvements, such as enhancing output resolution, speeding up training, and experimenting with advanced GAN variants like Wasserstein GANs or StyleGANs.